# COMS 4995-HW2

CHING-YI WANG

March 2026

## 1 Introduction

Predicting customer subscription to term deposits is a fundamental problem in financial decision-making, with direct implications for marketing efficiency and resource allocation. In this project, we formulate **term deposit subscription prediction as a binary classification problem** using the UCI Bank Marketing dataset. The objective is to predict whether a customer will subscribe to a term deposit based on demographic, financial, and campaign-related features.

The dataset contains both numerical and categorical variables, including age, job type, marital status, and previous marketing interactions. These mixed data types and non-linear relationships make the dataset well-suited for comparing different modeling paradigms.

This study focuses on two fundamentally different approaches: **Gradient Boosted Decision Trees (GBDT)** and **Multi-Layer Perceptrons (MLP)**. While GBDT models learn through sequential tree-based boosting, MLP models rely on gradient-based optimization in neural networks. The goal is to evaluate their performance, understand their learning behavior, and analyze their strengths and limitations in a controlled experimental setting.

## 2 Methods

### 2.1 Data Preprocessing

Initial inspection of the dataset revealed substantial missingness in several categorical variables. In particular, `poutcome` contained 25,867 missing values (81.7%), `contact` had 9,065 missing values (28.8%), `education` had 1,322 missing values (4.1%), and `job` had 203 missing values (0.64%). All other variables had negligible or no missing values.

The extremely high missing rate in `poutcome` suggests that this feature contains limited usable information and may introduce noise if naively imputed. Rather than removing a large portion of the dataset, missing categorical values were treated as a separate category (e.g., "unknown"), preserving sample size while allowing models to learn patterns associated with missingness itself. This approach is particularly appropriate for tree-based models, which can naturally handle such categories.

The dataset contains a mix of numerical and categorical features, requiring careful preprocessing to ensure compatibility across models. Missing values were handled using imputation strategies appropriate for each feature type. Numerical features were imputed using median values to reduce sensitivity to outliers, while categorical variables were imputed using the most frequent category.

Categorical variables were transformed using **one-hot encoding** to avoid imposing artificial ordinal relationships. This is particularly important for models such as MLP, which assume numerical input without inherent categorical structure.

In addition, the dataset was split into **training (70%), validation (15%), and test (15%)** sets to balance training stability and unbiased evaluation. All preprocessing steps, including imputation and encoding, were **fit on the training data only** and subsequently applied to validation and test sets to prevent data leakage.

For MLP models, feature scaling was applied using standardization:

$$z = \frac{x - \mu}{\sigma}$$

This step is necessary because neural networks rely on gradient-based optimization, which is sensitive to feature scale. Without scaling, features with larger magnitudes could dominate the learning process. In contrast, tree-based models such as GBDT are invariant to monotonic transformations and therefore do not require feature scaling.

The dataset is also imbalanced, with significantly fewer positive subscription cases. Therefore, evaluation metrics such as **F1-score and AUC-PR** were emphasized over accuracy.

### 2.2 Modeling steps

Two model families were implemented and compared under the same data split.

**Gradient Boosted Decision Trees (GBDT)** were implemented using XGBoost. This model builds an ensemble of decision trees sequentially, where each new tree corrects the residual errors of previous trees. Key hyperparameters were tuned using randomized search with cross-validation, including learning rate, tree depth, number of estimators, subsampling ratios, and regularization parameters (e.g., `reg_alpha`, `reg_lambda`, `gamma`, and `min_child_weight`). Additionally, `scale_pos_weight` was tuned to address class imbalance.

**Multi-Layer Perceptrons (MLP)** were implemented using scikit-learn. The MLP model consists of fully connected layers with non-linear activation functions, trained using backpropagation. Hyperparameters such as hidden layer size, activation function, learning rate, maximum iterations, and L2 regularization strength were tuned to optimize performance.

Both models were trained on identical data splits to ensure a fair comparison.

## 3 Results: Evaluation metrics and visualizations

Model performance was evaluated using accuracy, precision, recall, F1-score, and AUC-PR on the held-out test set. These metrics provide complementary perspectives, particularly in imbalanced classification settings where accuracy alone may be misleading. Table 1 summarizes the evaluation metrics for both models.

As shown in Table 1, XGBoost achieved higher recall (0.764) and F1-score (0.643), indicating a strong ability to identify positive cases. In contrast, the MLP model achieved higher precision (0.631) but lower recall (0.557), suggesting a more conservative prediction strategy that prioritizes correctness over coverage. This difference in behavior is further illustrated by the precision-recall curves in Figure 3.

Training efficiency was also evaluated by measuring total model training time during hyperparameter tuning. XGBoost required approximately 236 seconds, while the MLP model required approximately 155 seconds. Although XGBoost is generally considered computationally efficient, its larger hyperparameter search space and more complex tuning process resulted in longer training time in this setting. In contrast, the MLP model had a smaller search space, leading to faster overall tuning.

Training dynamics further highlight differences between the models. As shown in Figure 1, XGBoost demonstrates stable convergence, with validation performance plateauing as the number of trees increases, indicating effective control of overfitting. In contrast, the MLP model exhibits slower convergence, reflecting the challenges of optimizing neural networks and their sensitivity to hyperparameters. The relatively small gap between training and validation loss suggests effective regularization and controlled model complexity.

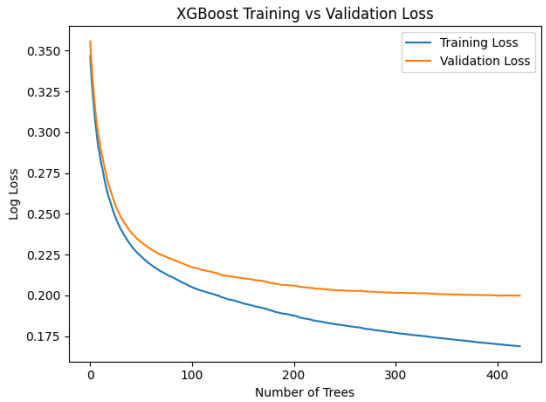


Figure 1: XGBoost Training vs. Validation Loss curve

To further analyze MLP performance, we examined the effect of network architecture on validation F1-score using cross-validation results (Figure 2). The results show that increasing model capacity improves performance, with the two-layer architecture (128, 64) achieving the highest validation F1-score. Simpler architectures such as (32,) and (64,) underperform, suggesting underfitting. This indicates that moderately deeper networks are better able to capture underlying patterns, although gains are not strictly linear with increasing complexity.

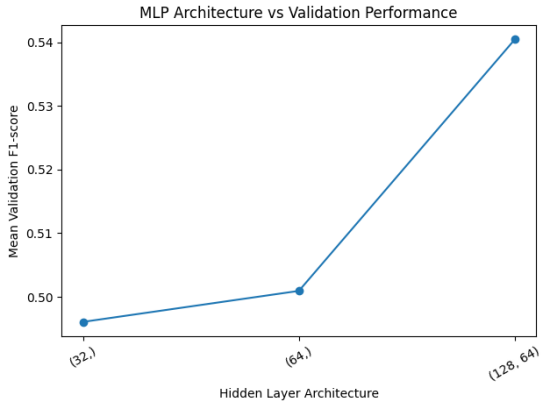


Figure 2: Effect of MLP Architecture on Validation Performance

Feature importance analysis from XGBoost identifies key predictors that contribute most to model decisions (Figure 4). In particular, `poutcome_success` emerges as the most influential feature, followed by `contact_missing` and `duration`, suggesting that prior campaign outcomes and interaction characteristics play a central role in prediction. This provides interpretability and insight into the underlying data structure, which is not directly available in MLP models.

Overall, the results suggest that XGBoost provides more balanced performance and better handling of class imbalance, while also offering greater interpretability compared to MLP.

| Model | Accuracy | Precision | Recall | F1 | AUC-PR | Time (sec) |
|---|---|---|---|---|---|---|
| XGBoost | 0.903 | 0.554 | 0.764 | 0.643 | 0.635 | 236.0 |
| MLP | 0.912 | 0.631 | 0.557 | 0.592 | 0.619 | 155.2 |

Table 1: Model Performance Comparison

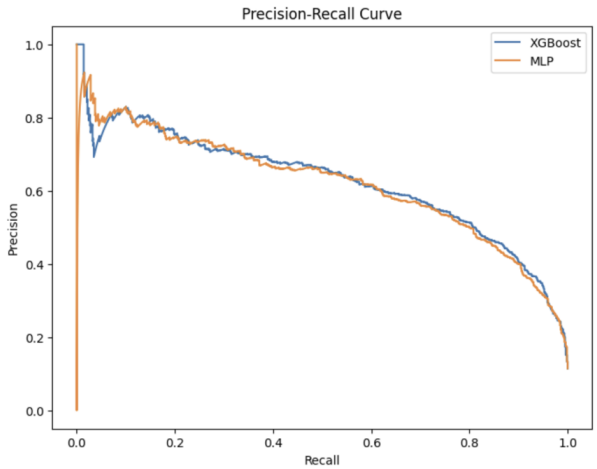


Figure 3: Precision-Recall Curve

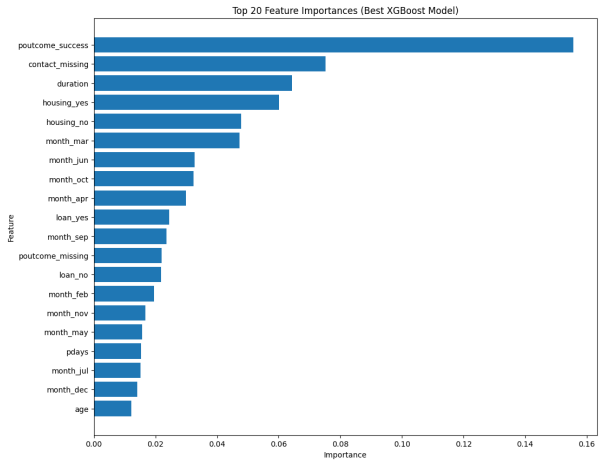


Figure 4: Top 20 Feature Importances

# 4 GBDT vs MLP Comparison: Side-by-side analysis with discussion

GBDT and MLP differ fundamentally in their learning mechanisms and practical behavior. GBDT models are more interpretable, as they provide feature importance measures and handle categorical variables effectively after encoding. They are also less sensitive to feature scaling and hyperparameter tuning. In contrast, MLP models require careful feature scaling and are more sensitive to hyperparameter choices such as learning rate and network architecture.

In terms of performance, XGBoost achieved higher recall and F1-score, making it more suitable for identifying positive cases in imbalanced datasets. MLP achieved higher precision but failed to capture as many positive instances, indicating a more conservative prediction behavior.

From a computational perspective, XGBoost required longer training time ($\approx$ 236 sec) due to the sequential construction of multiple decision trees, whereas the MLP model benefited from efficient vectorized operations during iterative optimization ($\approx$ 155 sec).

As shown in Figure 3, the XGBoost model maintains a higher recall across a wider range of precision levels, indicating stronger performance in identifying positive cases. In contrast, the MLP model achieves higher precision in certain regions but at the cost of significantly reduced recall. This trade-off highlights the differing behavior of the two models, with GBDT prioritizing coverage of positive cases and MLP favoring more confident but fewer positive predictions. Given the imbalanced nature of the dataset, recall and AUC-PR are particularly important, suggesting that XGBoost provides more robust overall performance.

In particular, `poutcome_success` and `duration` emerge as the most influential features, indicating that prior outcomes and interaction duration play a critical role in prediction.

GBDT and MLP also differ in how they handle categorical features and missing values. In this study, both models required categorical variables to be converted into numerical form using one-hot encoding. However, GBDT is generally more robust to sparse encoded features and does not require feature scaling, whereas MLP is more sensitive to the representation and scale of input variables. For missing values, both models relied on preprocessing before training. Missing categorical values were treated as an explicit "unknown" category, allowing the models to preserve sample size and potentially learn information carried by missingness itself. While both models used the same preprocessing pipeline for fairness, tree-based models are typically more tolerant of such irregularities than neural networks.

Overall, GBDT is more effective for structured tabular data, while MLP may be advantageous in high-dimensional or unstructured settings.

# 5 Evaluation and Interpretation

Model performance was evaluated using accuracy, precision, recall, F1-score, and AUC-PR on the held-out test set. These metrics provide complementary perspectives, particularly in imbalanced classification settings where accuracy alone may be misleading.

XGBoost achieved higher recall (0.764), F1-score (0.643), and AUC-PR (0.635), indicating strong ability to identify positive cases. In contrast, the MLP model achieved higher precision (0.631) and slightly higher accuracy (0.912), suggesting a more conservative prediction strategy that prioritizes correctness over coverage.

This trade-off is particularly important in imbalanced datasets, where recall and AUC-PR are often more informative than accuracy. The higher recall of XGBoost indicates that it is more effective at capturing positive cases, while MLP tends to miss more positives despite making more precise predictions.

Hyperparameter tuning further improved XGBoost performance. The best-performing configuration used 500 trees with a learning rate of 0.05, maximum depth of 4, subsample ratio of 0.7, column subsampling ratio of 0.8, `min_child_weight`=1, `gamma`=0, `reg_lambda`=1, and `scale_pos_weight`=3. This model achieved a best cross-validation F1-score of 0.631, indicating strong generalization performance. Overall, this configuration reflects a balance between model complexity and regularization, helping to prevent overfitting while maintaining predictive accuracy.

Figure 5 illustrates the effect of learning rate on model performance. Very small learning rates result in lower performance and higher variability, indicating underfitting and slow convergence. As the learning rate increases, performance improves and becomes more stable, with an optimal region around 0.05–0.1. Larger learning rates slightly degrade performance, suggesting increased variance and potential instability. Notably, the selected learning rate of 0.05 lies within this optimal region, confirming consistency between cross-validation results and empirical analysis.

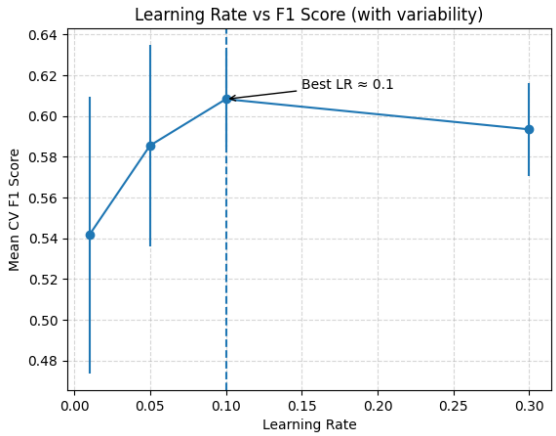


Figure 5: Effect of learning rate on XGBoost performance (mean CV F1-score with variability)

Figure 6 shows the training loss curve of the MLP model. The loss decreases rapidly during early iterations and gradually stabilizes, indicating effective learning and convergence. The smooth decline without divergence suggests that the model is well-tuned and does not exhibit severe overfitting or instability.

Figure 6: MLP training loss curve

Overall, while both models perform well, XGBoost demonstrates more balanced performance across metrics, particularly for the minority class, making it more suitable for this task.

### 5.1 Limitations and Bias-Variance Analysis

Several limitations exist in this study. First, hyperparameter tuning was limited to a predefined search space, and a more extensive or adaptive search (e.g., Bayesian optimization) may improve model performance.

Second, the dataset is imbalanced, which can bias models toward the majority class. Although evaluation metrics such as F1-score and AUC-PR were used to mitigate this issue, the observed differences between precision and recall suggest that both models are still influenced by class imbalance. In particular, XGBoost prioritizes recall, while MLP achieves higher precision, reflecting different trade-offs in handling minority class predictions.

From a bias-variance perspective, XGBoost's higher recall suggests lower bias in capturing the minority class, whereas the MLP's higher precision at the expense of recall indicates a more conservative decision boundary, potentially reflecting higher bias or greater sensitivity to variance in the imbalanced feature space. These behaviors are influenced by hyperparameter tuning via cross-validation, where model complexity and regularization were optimized, shaping the trade-off between bias and variance in each model. This highlights a fundamental trade-off between coverage and confidence in predictions across the two models.

Additionally, while GBDT models provide interpretable feature importance, they may not capture complex hierarchical feature representations as effectively as neural networks. Conversely, although MLP models are more expressive, their performance is more sensitive to architectural and optimization choices.

Overall, these results highlight that model selection should depend on the characteristics of the dataset, the importance of interpretability, and the desired balance between bias and variance.

## 6 AI Tool Usage Disclosure

1. **List AI tools used:** ChatGPT (OpenAI)
2. **Contribution of AI Tools:** I used ChatGPT to support coding efficiency and documentation clarity. Specifically, it assisted in generating starter code templates, debugging syntax errors, refining evaluation pipelines, and improving LaTeX formatting for the final report. The tool was also used to help structure performance summaries and enhance the clarity of technical explanations.
3. **Author Contributions:** All data preprocessing decisions, modeling choices, hyperparameter selections, performance interpretation, and model reasoning were independently performed by the author. The complete model implementation, feature engineering decisions, and result analysis were developed through the author's own understanding of machine learning principles. Furthermore, all AI-generated suggestions were carefully reviewed, modified, and validated to ensure correctness and alignment with assignment requirements.